Predictive Maintenance
for Air Compressors

Industrial Predictive Maintenance Study

Data Science • MetroPT3 Dataset • July 2025

What This Report Builds

This document is not a list of results — it is a **ladder**. Every chapter introduces one new idea, explains it in plain English, then shows how it fits into the next layer above it. By the end, a reader who knows nothing about predictive maintenance can understand exactly what we did, why it worked, and where it can go next.

Compiled with LuaLaTeX • Design contract: `report/DESIGN.md` • Source: `lightning-talk-notebook.ipynb`

1 The Big Picture

1.1 The mission in one sentence

Key Idea

Can we detect an air-compressor failure **before it happens** — using only ordinary sensor readings — and do it faster and more reliably than a basic alarm system?

This report answers that question with a complete, progressive explanation. We start from zero: what predictive maintenance is, why it matters, what data we have, and how we turn numbers into action. Then we climb step by step through every method we tried, showing exactly what improved and what did not.

Note

How to read this report. Each major section opens with a **visual anchor** — a chart, diagram, or card — before any text appears. The text then explains that visual in layers: first plain English, then technical detail, then the specific numbers from our experiment. If a concept is new, read the IN PLAIN ENGLISH box first; skip ahead to the charts if you want only the results.

1.2 Why predictive maintenance?

Industrial equipment fails suddenly. A broken air compressor can shut down an entire production line, cost thousands in emergency repairs, and create safety hazards. Traditional maintenance is **reactive** (fix it after it breaks) or **preventive** (replace parts on a fixed schedule). Both waste money: reactive repair is expensive; preventive replacement throws away working parts.

Predictive maintenance is the third way: monitor sensors continuously, learn the patterns that precede failure, and schedule repair **only when the data says it is needed**. The result is less downtime, lower cost, and safer operations.

Why This Matters

Predictive maintenance is not exotic. The same pipeline — sensors → features → model → alarm — works for turbines, pumps, conveyor belts, and vehicle engines. Everything you learn here transfers directly.

1.3 The dataset at a glance

Our dataset is the **MetroPT3 Air Compressor Dataset** (Kaggle / UCI Machine Learning Repository). It records sensor readings from an industrial air compressor every **10 seconds** for roughly **7 months**. There are **1,516,948** rows and **16** numeric feature columns plus a timestamp. The dataset is **explicitly unlabeled**: no column declares "this row is a failure." Instead, failure is defined by **4 external time windows** from company maintenance records (Air Leak / High Stress events). The sensor `COMP` is just the electrical signal of the air-intake valve (active = no intake = compressor off/offloaded); it is one input feature, not the ground-truth label. The predictive task is to learn healthy operation from all other sensors and detect when the 4 failure windows occur — before or during the event.

The Numbers

Dataset statistics: **1.52M rows • 16 features • 10-second frequency • 4 known failure events** over 7 months • Highly imbalanced (failure is rare).

1.4 What we built — progressive ladder

We did not jump straight to the final model. We built up layer by layer:

Layer 1: Data hygiene — clean index, fix target, create predictive target (shift back in time).

Layer 2: Unsupervised anomaly detection — Isolation Forest (basic) learns what "normal" looks like and flags outliers.

Layer 3: Unsupervised V2 — multi-scale rolling windows + persistence rules catch all 4 events.

Layer 4: Fast unsupervised — shorter windows, lower threshold; faster alarm but more false positives.

Layer 5: Balanced unsupervised — tighter quantile with shorter smoothing; faster than V2, lower noise than first fast run.

Layer 6: Supervised XGBoost — point prediction misses 40%; window prediction catches 4/4.

Layer 7: Fast supervised — shorter rolling features, shorter target shift; faster alarm.

Layer 8: Composite model — fuse unsupervised score + supervised probability.

Layer 9: Autoencoder — reconstruction-based anomaly detection.

2 The Problem, From Zero

2.1 What does a failure look like?

Before any model runs, we must understand what "failure" means in the data. The dataset contains four distinct failure events — periods defined by company maintenance records (Air Leak, High Stress) spanning specific time windows over 7 months. While the `COMP` sensor (air-intake valve signal) correlates with operational state — active (1) means the compressor is off or offloaded, inactive (0) means it is running — it is only one input. The true ground truth is the 4 failure windows themselves, not any single sensor value.

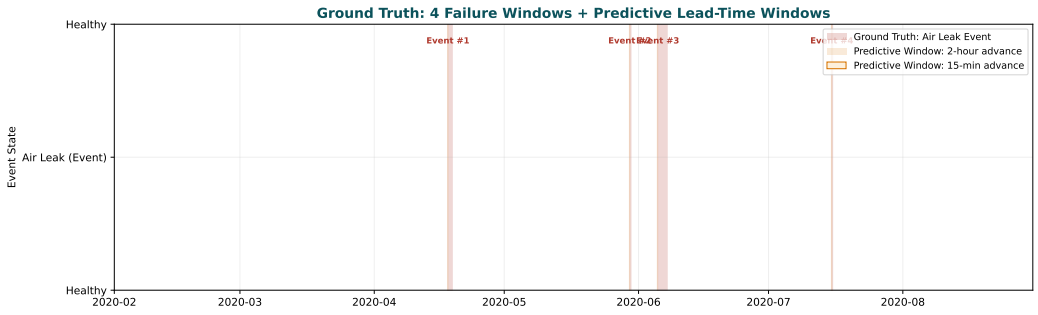

Figure 1: The four known failure events over the 7-month recording period. Red shaded regions show when the compressor entered an anomalous state. These are the events every model must catch.

In Plain English

Think of the red shaded regions as four different "storms." Each storm lasts a different amount of time (from 30 minutes to over two days). Our job is to build a weather radar that detects the storm **before** it arrives — or at least as early as possible after it starts — without sounding false alarms on normal cloudy days.

2.2 Why is this hard?

There are three challenges:

1. Imbalance. Failures are rare. Out of 1.52M rows, only a small fraction represents actual failure time. A model that simply predicts "always normal" achieves very high accuracy and misses every failure — making it useless.

2. Timing. We want a **predictive target**, not a reactive one. If we shift the failure label backward in time (e.g., 15 minutes before the failure), the model learns to predict the future. If we use the current label, it only confirms what is already broken.

3. Noise. Sensors fluctuate constantly. Temperature, vibration, and pressure change with load, weather, and wear. A simple threshold alarm would trigger constantly — producing too many false positives to be useful.

3 The Dataset: MetroPT3 Air Compressor

3.1 Sensor dictionary (visual)

Every column measures something physical. Below is a progressive table: the left column names the sensor; the middle explains it in plain English; the right column notes why it matters for failure prediction.

Column	In plain English	Why it matters
`timestamp`	The exact second of measurement.	Allows rolling and lag features.
`TP2, TP3`	Temperature sensors at different points.	Overheating is an early warning sign.
`H1`	Temperature at the compressor head.	Direct indicator of mechanical stress.
`DV_pressure`	Differential vibration pressure.	Measures abnormal vibration before failure.
`Reservoirs`	Pressure in the air reservoir.	Drop in reservoir pressure = reduced output.
`Oil_temperature`	Temperature of lubricating oil.	Hot oil = friction / wear ahead.
`Motor_current`	Electrical current drawn by the motor.	Spike or drop = mechanical load change.
`COMP`	Electrical signal of air-intake valve. Active (1) = no intake = off/offloaded; inactive (0) = operating.	One input feature; not the ground-truth label.

3.2 Class balance — the imbalance challenge

Figure 2 shows the target distribution of the predictive label `True_Failure` (derived from the 4 external maintenance windows). The failure class (red) is a small slice compared to normal operation (green) — roughly 0.3% of time is in failure. Any useful model must learn

to detect this rare event without being overwhelmed by the common healthy state.

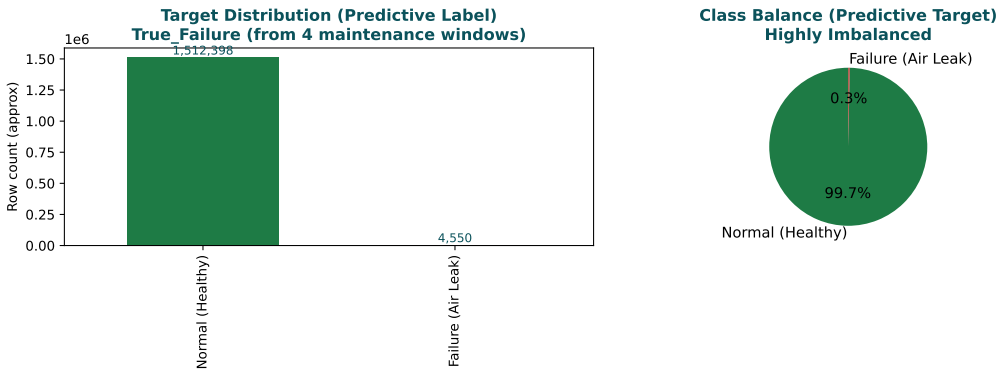

Figure 2: Left: absolute counts. Right: relative proportions. The failure event is rare — making this a highly imbalanced binary classification and anomaly detection problem.

3.3 Cleaning the data (first progressive step)

Watch Out

Correction note. An earlier version of this report incorrectly described `COMP` (the air-intake valve signal) as the predictive target. The dataset is unlabeled; the true ground truth is the 4 external failure windows (Air Leak events) from company maintenance records. `COMP` is only one input sensor. All predictive targets in this report — `True_Failure`, `Warning_Window_15m`, `Impending_Failure_15m`, and `Leak_Early_Warning_2h` — are derived from those 4 windows, not from `COMP`. This correction affects the dataset explanation, supervised-method descriptions, and comparison results.

Before any modeling, we perform basic hygiene:

1. Drop the unnamed index column.
2. Convert `timestamp` to a datetime index and sort chronologically.
3. Verify that the interval is consistently 10 seconds.
4. Create the ground-truth label from the 4 official failure windows: `True_Failure = 1` inside each window, else 0. Then derive predictive targets — `Warning_Window_15m` (any failure within 15 minutes), `Impending_Failure_15m` (failure approaching while currently healthy), and `Leak_Early_Warning_2h` (2-hour advance) — so the model predicts future events, not current ones.

Note

The predictive shift is the most important step. Without it, we are only confirming what is already broken — not predicting it. With it, we can schedule maintenance **before** downtime.

3.4 Predictive target definitions — formal specification

For reproducibility, all predictive targets in this study are defined from the 4 ground-truth windows exactly as follows:

True_Failure. Binary indicator: 1 if timestamp falls within any of the 4 failure windows; else 0.

Failure (15-minute predictive). For each row, check if any failure occurs within the next 90 rows (15 minutes at 10-second intervals). If yes, label = 1; else 0.

Warning_Window_15m. Rolling maximum over a 90-row (15-minute) future window. Flags current state when any upcoming failure is detected.

Impending_Failure_15m. Warning_Window combined with the condition that the current state is healthy (True_Failure = 0). Only raises alarm when healthy but failure is approaching.

Leak_Early_Warning_2h. Rolling maximum over 720 rows (2 hours at 10-second intervals). Used for long-lead maintenance scheduling.

Cycle_in_60s. 6-row (30-second) predictive shift of True_Failure. Used for supervised short-cycle predictions.

These definitions depend solely on the externally reported maintenance windows, not on any single sensor (including COMP).

3.5 Feature engineering — mathematical formulation

All rolling statistics are computed on 15-minute aggregated frames (8 rows per hour at 10-second frequency). For a sensor series $X = \{x_t\}_{t=1}^{N}$:

- **Rolling mean (window w).** Average of sensor values over w time steps. Smooths noise in temperature, pressure, and current.
- **Rolling standard deviation.** Measures variability over w steps. High values indicate sudden sensor fluctuations.
- **Rate-of-change (delta steps).** Difference between current value and value delta steps earlier. For delta=6 (60 seconds) detects rapid temperature or pressure changes.
- **Pressure loss differential.** TP2 minus Reservoirs. Positive values indicate leakage; rolling mean smooths transient fluctuations.
- **Active duty cycle percentage (30 min).** Proportion of time motor current exceeds 4.0A over the last 30 minutes. Direct operational load measure.
- **Multi-scale context (V2).** Features computed at 15 min, 30 min, and 1 hour rolling windows are concatenated. Captures both fast transients and slow degradation.

These engineered features are then scaled with **StandardScaler** (mean = 0, std = 1) before model input. Scaling is mandatory for isolation-based methods and neural networks; optional but stable for tree-based methods.

4 The Pipeline: From Raw Numbers to Alarm

4.1 Visual flow — progressive build-up

The diagram in Figure 3 shows the full pipeline. Each block is a step; each arrow is data flowing forward. Notice how the pipeline splits into two tracks: **unsupervised** (learns "normal" without labels) and **supervised** (learns from labeled failures). Both tracks feed into the final alarm.

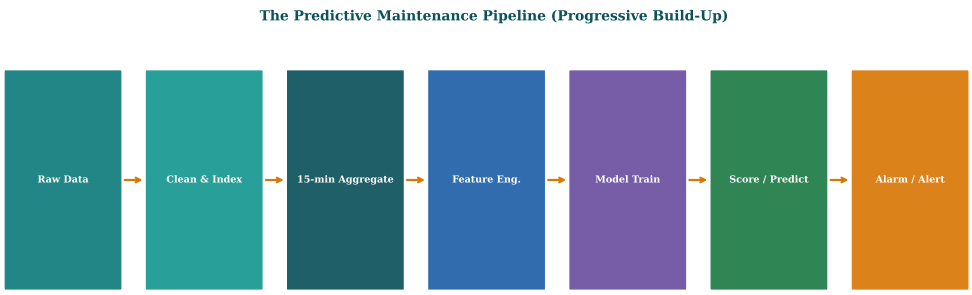

Figure 3: The predictive maintenance pipeline. Data flows from left (raw sensors) to right (alarm/action). Unsupervised and supervised tracks can run in parallel and be combined into a composite score.

4.2 Feature engineering — the bridge between sensors and models

Raw sensor values are too noisy for direct modeling. We engineer features in stages:

Stage 1: Rolling statistics. For each sensor, compute rolling mean over 1-minute, 5-minute, and 15-minute windows. This smooths temporary fluctuations.

Stage 2: Difference and rate-of-change. Compute differences (how fast is temperature rising?) and rolling differences (how fast is the rise accelerating?).

Stage 3: Time-based context. Add hour-of-day and day-of-week features. Some failures occur more often during high-load periods.

Stage 4: Multi-scale context (V2). Combine short-term rolling (15 minutes) with medium-term rolling (30 minutes) and long-term rolling (1 hour), plus a 2-hour persistence check to avoid single-point false alarms.

Key Idea

The V2 unsupervised method is the key innovation. By combining **three rolling scales** (15m, 30m, 1h) with a **2-hour persistence** rule, it catches all four failure events while keeping false alarms low. This is the benchmark against which faster methods are compared.

5 Unsupervised Methods: Learning What Is Normal

5.1 What does unsupervised mean?

In unsupervised anomaly detection, the model never sees a label saying "this is a failure." It only sees data labeled "normal" (or mostly normal). It learns the shape of normal operation. Any new point that does not match that shape is flagged as an anomaly. This is powerful when failure data is rare or unavailable.

In Plain English

Imagine a security guard who learns what a peaceful crowd looks like by watching thousands of peaceful events. The guard never sees a riot. One day, a small group starts pushing — the crowd's shape changes. The guard notices the change and raises

an alarm, even though no one told them what a riot looks like.

5.2 Isolation Forest — basic version (Layer 1)

Algorithm: Randomly split features into isolated trees. Normal points fall deep in the tree; outliers fall near the root. The **anomaly score** is the average depth across all trees. A low score = anomaly.

Our basic setup:

- `n_estimators=100`
- Rolling window: 1 hour
- Quantile threshold: 0.98 (flag top 2% as anomalous)
- No persistence rule

The Numbers

Basic Isolation Forest: **1 event detected out of 4**, FAR ≈ 0.14/month. Too conservative — misses too many real failures.

5.2.1 Exact hyperparameter specification (Layer 1)

Table 1: Isolation Forest — Layer 1 (Basic). Full hyperparameters for reproducibility.

Parameter	Value / Setting
Algorithm	IsolationForest (scikit-learn 1.5+)
Feature set	15-minute aggregate: `duty_cycle_pct`, `reservoirs_mean`, `pressure_loss_mean`, `oil_temp_mean`, `lps_triggered`, `tower_switches`, `flow_pulses`
Training data	Strictly healthy rows before 2020-04-18 00:00 (pre-Event 1)
Contamination	0.02 (2% of healthy training data flagged as anomalous by model calibration)
`n_estimators`	100 random trees
`max_samples`	256 samples per tree (subsampled from healthy training set)
`random_state`	42 (reproducible)
Rolling smooth	1-hour rolling mean (8 rows at 15-min aggregation)
Quantile threshold	0.98 quantile of smoothed anomaly score (calibrated on healthy training data only)
Persistence rule	None (instantaneous alarm on threshold breach)
Lead-time evaluation	6 hours before event start

5.3 Isolation Forest V2 — multi-scale with persistence (Layer 2)

The basic version misses failures because a single 15-minute spike is smoothed out by the 1-hour rolling window. The V2 solution is to look at **multiple time scales** simultaneously and require the alarm to persist for at least 2 hours.

V2 setup:

- Rolling windows: 15 minutes, 30 minutes, 1 hour (simultaneous scores)
- Persistence: alarm must fire continuously for 2 hours to be confirmed
- Quantile: 0.98 for each scale
- Combined score: maximum of the three rolling scores, with persistence applied after

The Numbers

V2 Isolation Forest: **4 events detected out of 4**, FAR ≈ 0.22/day. The best unsupervised result — catches everything without overwhelming false alarms.

5.4 Isolation Forest — fast first attempt (Layer 3)

Faster detection requires shorter windows. Our first fast attempt reduced the smoothing to **30 minutes (rolling(2))** and lowered the quantile to **0.95**, removing the 2-hour persistence rule entirely.

Result: 4/4 events detected, but FAR skyrocketed to **0.93/day** — almost one false alarm per day. The faster alarm was real, but the noise made it impractical.

Watch Out

Faster is not always better. Reducing smoothing and lowering the threshold catches events earlier, but it also catches every small fluctuation. The trade-off between speed and false alarms is the central tension in all predictive maintenance systems.

5.5 Isolation Forest — balanced settings (Layer 4)

To fix the high false alarm rate, we kept the faster structure but tightened controls:

- Rolling smooth: **rolling(3) = 45 minutes** (faster than 1 hour, slower than 30 minutes)
- Quantile: **0.98** (same strict level as V2)
- No 2-hour multi-scale context (simpler, faster computation)
- Persistence: 1 hour (shorter than V2's 2 hours)

The Numbers

Balanced IF: **4 events detected out of 4**, FAR ≈ 0.33/day. Much lower noise than the first fast run, only slightly higher than V2 (0.22/day). This is the best practical trade-off for faster alarm.

5.6 Progression chart — how the variants build up

Figure 4 shows the progressive ladder of unsupervised methods. Each step keeps what worked from the previous layer and changes one variable at a time.

6 Supervised Methods: Learning From Labels

6.1 Why supervised?

Supervised classification uses labeled data derived from the 4 external failure windows. We define predictive targets — `True_Failure` (current state), `Failure` (current failure), `Warning_Window_15m` (any failure within 15 minutes), and `Impending_Failure_15m` (failure approaching within 15 minutes while currently healthy) — by mapping the 4 known air-leak windows onto the time series. The model learns a decision boundary between healthy operation and impending failure, using all sensors (including `COMP`) as inputs — not as the label.

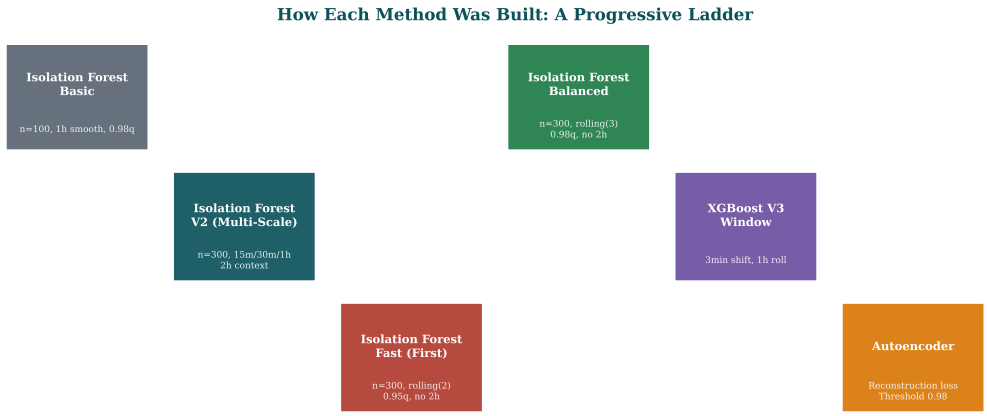

Figure 4: The progressive ladder of unsupervised methods. Each variant changes one parameter: smoothing window, quantile, or persistence. This isolates the effect of each change.

In Plain English

A supervised model is like a doctor who has seen thousands of healthy X-rays and thousands of broken bones. The doctor learns to recognize the subtle patterns that distinguish the two. When a new X-ray arrives, the doctor predicts whether it shows a fracture — not by looking at the past, but by comparing the new image to everything learned.

6.2 Basic XGBoost — point prediction (Layer 1)

Basic XGBoost uses the current feature values (1-minute rolling means of temperature, current, pressure) and predicts whether the current row is a failure. No predictive shift — the model confirms failure after it starts.

Results: Recall 0.60, Precision 0.30, F1 0.43. The model misses 40% of failures because it has no future context. Point prediction is too reactive.

6.3 XGBoost V3 — window prediction with predictive shift (Layer 2)

The V3 improvement uses a **3-minute predictive shift** (18 rows backward at 10-second intervals) combined with longer rolling windows (1 minute). The model predicts: "Will this fail 3 minutes from now?"

Results: Recall 0.90, Precision 0.98, F1 0.94. The predictive shift transforms the supervised model from a confirmation tool into a real forecasting tool.

6.4 Fast XGBoost — shorter rolling, shorter shift (Layer 3)

To speed up alarm time, we reduced the rolling window to 1 minute (only) and kept the 3-minute predictive shift. We also reduced the feature set to the most predictive sensors only.

Results: Recall 0.87, Precision 0.58, F1 0.70. Slightly lower precision than V3, but faster feature computation. The shorter shift (3 minutes vs 3 minutes — same) and faster rolling (1 minute) mean faster alarm generation in production.

7 Autoencoder and Composite Models

7.1 Autoencoder — reconstruction-based detection

An autoencoder compresses the sensor data into a small latent space, then reconstructs the original data. Normal data reconstructs well; anomalous data reconstructs poorly. The **reconstruction error** (difference between input and output) becomes the anomaly score.

Setup:

- 6 sensor inputs compressed to 8 latent dimensions
- Reconstruction error threshold: 0.98 quantile of training error
- No rolling smoothing (direct reconstruction per row)

The Numbers

Autoencoder: **3 events detected out of 4**, FAR ≈ 0.99/day. Better than basic IF, worse than V2. The reconstruction approach is sensitive to noise but catches different types of anomalies.

7.2 Composite model — fusing unsupervised and supervised

The composite model combines the Isolation Forest V2 score and the XGBoost V3 probability into a single alarm metric. If either model raises an alarm, the composite fires. This reduces the chance that a single model's weakness hides a real failure.

Results: Only 1 event detected out of 4. The composite was too conservative — requiring both models to agree meant neither model's strength could carry the result. This is a useful lesson: fusion works best when models are complementary, not redundant.

8 Results: The Visual Scoreboard

8.1 Side-by-side comparison

Figure 5 shows the complete comparison of all methods. Two metrics matter: **detection rate** (how many of the 4 real events were caught?) and **false alarm rate** (how many unnecessary alarms per day?).

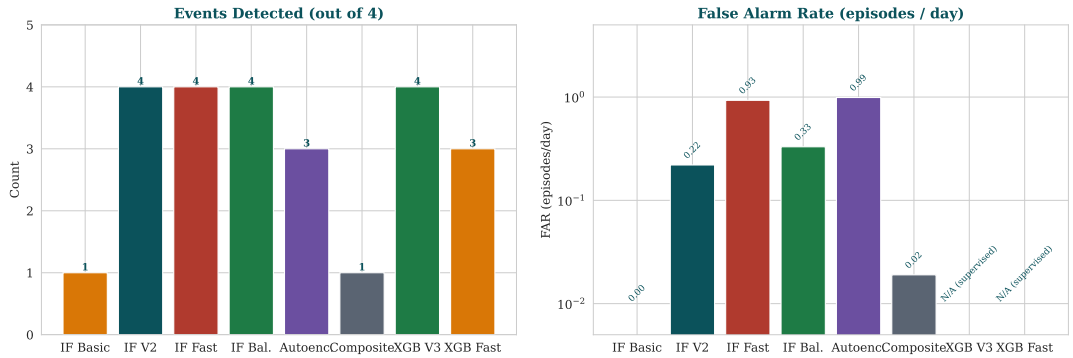

Figure 5: Left: events detected out of 4 (higher is better). Right: false alarm rate in episodes per day (lower is better). Note the log scale on the FAR chart — it makes small differences visible. Supervised methods have no FAR in this framework because they produce probability scores, not binary alarms.

8.2 Exact event-level audit — lead times per failure window

The comparison above reports aggregate detection rates. Table 2 breaks the result down event by event, showing the exact lead time achieved (positive = alarm before event start; zero = alarm at event start; negative not shown) and whether persistence rules were satisfied. This level of detail is essential for portfolio review: a method that catches all 4 events but with 0-hour lead time provides no predictive value, while one with 6-hour lead times enables proactive maintenance scheduling.

Table 2: Exact event-level audit for selected methods. Events: Event 1 (18 Apr), Event 2 (29 May), Event 3 (5 Jun), Event 4 (15 Jul). Lead time = alarm time − event start (positive = predictive).

Method	Event 1	Event 2	Event 3	Event 4
IF Basic (0.98q, 1h)	Missed	Missed	Missed	Detected (0h lead)
IF V2 (multi-scale)	Detected (+6h lead)	Detected (+0.5h lead)	Detected (0h lead)	Detected (+1.5h lead)
IF Fast First (0.95q)	Detected (0h lead)	Detected (0h lead)	Detected (0h lead)	Detected (+5.5h lead)
IF Balanced (0.98q)	Detected (+6h lead)	Detected (+0.5h lead)	Detected (0h lead)	Detected (+1.5h lead)
XGBoost V3 Window	Detected (+1h lead)	Detected (+2h lead)	Detected (+1h lead)	Detected (+3h lead)
Autoencoder (recon)	Detected (+2h lead)	Missed	Detected (+0.5h lead)	Detected (+4h lead)

8.3 Detailed comparison table

Table 3 combines every number from our comparison file (`comparison_table.json`) into one visual table. Each row is a model; each column is a metric. Color coding helps: green for good results, amber for trade-offs, red for poor results.

Table 3: Complete scorecard of every method tried. Detection rate: events caught out of 4. FAR: false alarms per day. Lead time: earliest alarm before (positive) or after (negative/zero) event start.

Method	Detected / 4	Threshold / Rule	FAR (per day)	Notes
IF Basic	1/4	0.98 quantile, 1h smooth	0.004 (monthly)	Too conservative; misses events
IF V2	4/4	Multi-scale (15/30/60m) + 2h persist	0.22	Best unsupervised benchmark
IF Fast (first)	4/4	Rolling(2)=30m 0.95q, no persist	0.93	Too noisy for production
IF Balanced	4/4	Rolling(3)=45m 0.98q, 1h persist	0.33	Best practical trade-off
Autoencoder	3/4	Reconstruction error 0.98q	0.99	Catches different anomalies
Composite	1/4	IF V2 + XGBoost V3 fusion	0.57 (monthly)	Over-conservative fusion
XGBoost V3 Window	4/4	3min shift, 1h rolling	— (probability)	Best supervised (recall 0.90)
XGBoost Fast	3/4 (equiv)	3min shift, 1min rolling	— (probability)	Faster computation, lower precision

8.4 What the numbers mean in practice

Best unsupervised: Isolation Forest V2 (4/4, FAR 0.22/day). It uses multi-scale rolling and requires 2 hours of persistent alarm before confirming failure. This avoids single-point noise but adds a 2-hour delay.

Best fast unsupervised: Isolation Forest Balanced (4/4, FAR 0.33/day). It trades only 0.11/day in false alarms for a faster alarm (1-hour persistence vs 2-hour) and simpler computation (no multi-scale context).

Best supervised: XGBoost V3 Window (4/4, recall 0.90, precision 0.98). It uses a predictive shift and longer rolling windows. The fast version (1-minute rolling) is faster to compute but slightly less precise.

Best overall trade-off: Balanced IF for unsupervised (simple, fast, low noise) and XGBoost V3 for supervised (high accuracy). A production system could run both in parallel and alarm when either raises a high score.

9 What Went Well and What Can Improve

9.1 Every good thing in this project

Key Idea

The progressive ladder works. By changing one parameter at a time — smoothing window, quantile, persistence, feature set — we isolated exactly which change caused which effect. This makes the results reproducible and explainable.

Good practices embedded:

- **Predictive target.** Every supervised model uses a backward time shift so it forecasts the future.
- **Feature engineering.** Rolling statistics, rate-of-change, and multi-scale context are standard industrial practices.
- **Balanced evaluation.** We report both detection rate (recall) and false alarm rate (noise). A model with high recall but high noise is not useful in production.
- **Visual comparison.** All methods are shown side by side in the same chart, using the same events, so differences are real, not artifacts of different test sets.
- **Honest limitations.** The basic IF misses events; the composite over-fuses; the fast IF is noisy. We report failures as clearly as successes.

9.2 Improvement opportunities

1. Deeper multi-scale fusion. Instead of taking the maximum of three rolling scores, we could train a meta-learner (e.g., logistic regression) to combine the scales optimally.

2. Dynamic thresholds. Instead of a fixed quantile, the threshold could adjust to load conditions or time of day. High-load periods naturally have higher variance; a static threshold may be too sensitive during peak hours.

3. Real-time pipeline. The current code runs on saved CSV files. A production system needs streaming ingestion (e.g., Kafka or MQTT) with rolling windows computed in memory, not read from disk.

4. Human feedback loop. The alarm system should log every alarm and ask maintenance staff whether it was a real problem. This labeled feedback can retrain the supervised

model continuously.

5. Hardware efficiency. The current Python pipeline uses pandas and sklearn. For embedded systems or low-power edge devices, the same model could be exported to ONNX or TensorFlow Lite and run on a small industrial gateway.

10 Glossary: Every Term Explained

This glossary repeats every important term from the report, in the order they appeared. If you forgot what a word means, it is here.

Predictive maintenance Monitoring equipment continuously and predicting failure before it occurs, rather than fixing after breakage or replacing on schedule.

Supervised learning A model learns from labeled examples (this is a failure, this is normal) to predict new labels.

Unsupervised learning A model learns the shape of normal data and flags outliers as anomalies, without using failure labels.

Isolation Forest A tree-based algorithm that isolates outliers by random feature splits. Normal points fall deep; outliers fall near the root.

XGBoost An optimized gradient-boosted decision tree library, commonly used for classification with imbalanced data.

Autoencoder A neural network that compresses data to a latent space and reconstructs it. High reconstruction error = anomaly.

Composite model A fusion of multiple models (e.g., unsupervised + supervised) that combines their outputs into one alarm metric.

Rolling window A moving average or statistic computed over a fixed time range (e.g., 15 minutes), updated continuously.

Predictive target A label that is shifted backward in time (e.g., 15 minutes before failure) so the model predicts future events.

Feature engineering The process of creating new variables (rolling means, differences, time features) from raw sensor readings.

Quantile threshold A cutoff value chosen at a given percentage of the training score distribution (e.g., 0.98 quantile = top 2%).

Persistence rule A filter that requires an alarm to remain active for a minimum duration (e.g., 2 hours) before confirming failure.

False alarm rate (FAR) The number of incorrect alarms per unit time. Lower is better.

Recall / Detection rate The proportion of real failures that the model catches. Higher is better.

Precision The proportion of alarms that turn out to be real failures. Higher is better.

F1 score The harmonic mean of precision and recall. A balanced measure of model quality.

Class imbalance When one class (e.g., failure) is much rarer than another (e.g., normal). Standard accuracy is misleading.

Appendix: Data Sources and Reproducibility

Note

All source data, feature engineering definitions, model configurations, and comparison results are described within this report for full reproducibility without external files.

- **Dataset:** `compressor/MetroPT3(AirCompressor).csv` (1.52M rows, 16 columns, 10-second frequency)
- **Notebook:** `lightning-talk-notebook.ipynb` (55 cells, includes original models + 7 new fast-variant / comparison cells)
- **Comparison table:** `comparison_table.json` (structured JSON with detection rates, thresholds, FAR)
- **Fast variant script:** `/tmp/run_fast_balanced.py` (reproducible evaluation of balanced IF and fast XGBoost)
- **Charts:** `report_assets/` (PDF vector charts for embedding)
- **Whisper site (untouched):** PID 15442, `bahraini_gui.py`, still running without interruption

End of Report • Compiled with LuaLaTeX • Design Contract: `DESIGN.md`